

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using `fork()`. 3. Execute the command in the child process using an appropriate `exec()` system call. 4. Allow the parent process to wait for the child using `wait ()`. 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (`uname`, `lscpu`, `lsblk`, `ps`, `top`), investigate the relationship between hardware resources and operating system services.

Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

exp1.c

```
#include <stdio.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <stdlib.h>


int main()
{
    char command[100];

    printf("Enter a Linux command: ");
    scanf("%s", command);

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
```

```
        return 1;
    }
    else if (pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        execlp(command, command, NULL);

        perror("exec failed");
        exit(1);
    }
    else
    {
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}
```

Output :

```

naveen123@LAPTOP-4NVQJ2C7:~$ nano exp1.c
naveen123@LAPTOP-4NVQJ2C7:~$ gcc exp1.c -o exp1
naveen123@LAPTOP-4NVQJ2C7:~$ ./exp1
Enter a Linux command: ls
Parent Process
Parent PID: 700
Child PID: 701
Child Process
Child PID: 701
Parent PID: 700
OSSP      exp1  fork.c      input      main      progeight.c  progfour    prognine.c  progseven   progsix.c   progtwo    sell.c
command_executor  exp1.c  fork_process  input.c    main.c    progfive     progfour.c  progone     progseven.c  progtthree  progtwo.c
command_executor.c  fork   fork_process.c  input.txt  progeight  progfive.c   prognine    progone.c   progsix     progtthree.c  sell
Child process completed.
naveen123@LAPTOP-4NVQJ2C7:~$ |

```

Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

exp2.c

```

#include <stdio.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <stdlib.h>

int main()
{
    pid_t pid;

    printf("Before fork():\n");

    printf("Process PID : %d\n", getpid());

```

```
printf("Parent PPID : %d\n", getppid());
```

```
pid = fork();
```

```
if (pid < 0)
```

```
{
```

```
    perror("fork failed");
```

```
    return 1;
```

```
}
```

```
else if (pid == 0)
```

```
{
```

```
    // Child process
```

```
    printf("\n--- CHILD PROCESS ---\n");
```

```
    printf("Child PID : %d\n", getpid());
```

```
    printf("Child PPID : %d\n", getppid());
```

```
    printf("Child State : Running\n");
```

```
    sleep(5);
```

```
    printf("Child State : Terminating\n");
```

```
    exit(0);
```

```
}
```

```
else
```

```
{
```

```
    // Parent process
```

```
    printf("\n--- PARENT PROCESS ---\n");
```

```
    printf("Parent PID : %d\n", getpid());
```

```
    printf("Parent PPID : %d\n", getppid());
```

```

printf("Child PID   : %d\n", pid);

printf("Parent State : Running\n");


printf("\nParent waiting for child...\n");

wait(NULL);


printf("Child terminated.\n");

printf("Parent State : Running\n");

printf("Parent State : Terminated after completion\n");
}

return 0;
}

```

Output :

```

naveen123@LAPTOP-4NVQJ2C7:~$ nano fork_process.c
naveen123@LAPTOP-4NVQJ2C7:~$ gcc fork_process.c -o fork_process
naveen123@LAPTOP-4NVQJ2C7:~$ ./fork_process
Before fork():
Process PID   : 566
Parent PPID   : 492

--- PARENT PROCESS ---
Parent PID    : 566
Parent PPID   : 492
Child PID     : 567
Parent State  : Running

Parent waiting for child...

--- CHILD PROCESS ---
Child PID     : 567
Child PPID    : 566
Child State   : Running
Child State   : Terminating
Child terminated.
Parent State  : Running
Parent State  : Terminated after completion
naveen123@LAPTOP-4NVQJ2C7:~$ |

```

Develop a C program using `fork()` that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as `ps`, `top`, and `/proc`. Document the observations.

exp3.c

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
int main()
```

```
{
```

```
    pid_t pid;
```

```
    printf("Before fork()\n");
```

```
    printf("Process PID: %d\n", getpid());
```

```
    pid = fork();
```

```
    if (pid < 0)
```

```
    {
```

```
        perror("fork failed");
```

```
        return 1;
```

```
    }
```

```
    if (pid == 0)
```

```
{  
    printf("\nChild Process\n");  
    printf("PID : %d\n", getpid());  
    printf("PPID : %d\n", getppid());  
  
    sleep(10);  
  
    printf("Child process terminating...\n");  
}  
else  
{  
    printf("\nParent Process\n");  
    printf("PID : %d\n", getpid());  
    printf("PPID : %d\n", getppid());  
  
    sleep(5);  
  
    printf("Parent process terminating...\n");  
}  
  
return 0;  
}
```

Output:

```

naveen123@LAPTOP-4NVQJ2C7:~$ nano exp3.c
naveen123@LAPTOP-4NVQJ2C7:~$ gcc exp3.c -o exp3
naveen123@LAPTOP-4NVQJ2C7:~$ ./exp3
Before fork()
Process PID: 916

Parent Process
PID : 916
PPID : 492

Child Process
PID : 917
PPID : 916
Parent process terminating...
naveen123@LAPTOP-4NVQJ2C7:~$ Child process terminating...

```

4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

exp4.c

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    int i;
```

```
    pid_t pid;
```

```
    for (i = 0; i < 3; i++)
```

```
    {
```



```
pid = fork();

if (pid < 0)
{
    perror("fork failed");
    exit(1);
}

if (pid == 0)
{
    printf("Child %d: PID = %d, PPID = %d\n",
        i + 1, getpid(), getppid());

    sleep(i + 1);

    printf("Child %d completed.\n", i + 1);
    exit(0);
}

for (i = 0; i < 3; i++)
{
    wait(NULL);

    printf("Parent collected a child.\n");
}
```

```
printf("All children completed.\n");
```

```
return 0;
```

```
}
```

```
naveen123@LAPTOP-4NVQJ2C7:~$ nano exp4.c
naveen123@LAPTOP-4NVQJ2C7:~$ gcc exp4.c -o exp4
naveen123@LAPTOP-4NVQJ2C7:~$ ./exp4
Child 1: PID = 976, PPID = 975
Child 2: PID = 977, PPID = 975
Child 3: PID = 978, PPID = 975
Child 1 completed.
Parent collected a child.
Child 2 completed.
Parent collected a child.
Child 3 completed.
Parent collected a child.
All children completed.
naveen123@LAPTOP-4NVQJ2C7:~$ |
```